DI во пракса

@Autowired, слоеви на апликацијата, повеќе имплементации

1 Час 1 — 2 Час 2 — **3 Час 3** — 4 Час 4

Цели на часот

Потсетник

- IoC: Spring ги креира објектите
- Бин = објект управуван од Spring
- `@Component`, `@Service`, `@RestController`
- Spring Boot: стартери, вграден сервер
- Грешка: недостасува бин

Денес ќе научиме

1. `@Autowired` и **трите видови вметнување**
2. Кога `@Autowired` **мора** да се напише
3. Слоеви: **Controller → Service → Repository**
4. Два бина од ист тип: `@Primary`, `@Qualifier`, име
5. Регистрирање бинови со `@Bean`

@Autowired: „Spring, тука вметни бин“

1 · Конструктор

```
@Autowired
NotificationService(
    NotificationSender s) {
  this.sender = s;
}
```

препорачано

2 · Setter (метод)

```
@Autowired
void setSender(
    NotificationSender s) {
  this.sender = s;
}
```

опционални зависности

3 · Поле

```
@Autowired
NotificationSender sender;
```

избегнувај во апликациски код

Spring бара бин **по тип** (`NotificationSender`) и го вметнува на означеното место
Пакет: `org.springframework.beans.factory.annotation.Autowired`

Три начини за вметнување

Преку конструктор

Зависностите се параметри на конструкторот

Препорачано — користиме секаде

Преку setter

Spring го повикува setter методот по креирањето

За **опционални** зависности

Преку поле

`@Autowired` директно на полето

Избегнувај во нов код

1 · Преку конструктор: со и без @Autowired

Експлицитно

```
@Service
public class NotificationService {
    private final NotificationSender sender;

    @Autowired
    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }
}
```

Еден конструктор → @Autowired не е потребен

```
@Service
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }
}
```

Двата примери работат исто. Кога има **еден** конструктор, Spring автоматски го користи (Spring 4.3+).

Повеќе конструктори: @Autowired е задолжителен

```java
@Service
public class NotificationService {
    private final NotificationSender sender;
    private final String prefix;

    @Autowired                          // ← Spring го користи овој
    public NotificationService(NotificationSender sender) {
        this(sender, "[FINKI] ");
    }

    public NotificationService(NotificationSender sender,
                               String prefix) {
        this.sender = sender;
        this.prefix = prefix;
    }
}
```

- Два конструктори → Spring не знае кој да го повика
  - Еден конструктор за продукција, друг конструктор за тестирање
- `@Autowired` го означува вистинскиот
- Без `@Autowired` Spring бара празен конструктор → грешка

Правило: Еден конструктор = без `@Autowired` · повеќе конструктори = `@Autowired` на еден

Повеќе конструктори: @Autowired е задолжителен

```java
@Service
public class NotificationService {
    private final NotificationSender sender;
    private final Clock clock;

    @Autowired                                              // Spring го користи овој
    public NotificationService(NotificationSender sender, Clock clock) {
        this.sender = sender;
        this.clock = clock;
    }

    public NotificationService(NotificationSender sender) {       // за рачно креирање
        this(sender, Clock.systemDefaultZone());                  // стандардна вредност
    }

    public void notifyUser(String to, String message) {
        sender.send(to, "[" + LocalTime.now(clock) + "] " + message);
    }
}

@Bean Clock clock() { return Clock.systemDefaultZone(); }         // во @Configuration
var service = new NotificationService(new ConsoleEmailSender());  // рачно, без Spring
```

- Два конструктори → Spring не знае кој да го повика
- `@Autowired` → овој го користи Spring (`Clock` е бин)
- Пократкиот → за рачно креирање со `new`
- **Тест:** истиот конструктор со `Clock.fixed(…)`
- Без `@Autowired`: грешка „No default constructor found“

Правило: Еден конструктор = без `@Autowired` · повеќе конструктори = `@Autowired` на еден

2 · Преку setter

```java
@Service
public class NotificationService {
    private NotificationSender sender;     // не може final

    @Autowired
    public void setSender(NotificationSender sender) {
        this.sender = sender;
    }

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}
```

- Spring го креира објектот, па **сам** го повикува setter-от
- Без `@Autowired` setter-от **не** се повикува → `sender` е `null`
- Полето не може да биде `final`
- Корисно за **опционални** зависности

3 · Преку поле

❌ Погрешен начин

```
@Service
public class NotificationService {

    @Autowired
    private NotificationSender sender;

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}

// во unit тест без Spring:
var service = new NotificationService();
service.notifyUser("ana@finki.ukim.mk", "Здраво");
// NullPointerException — sender == null
```

- Spring го поставува приватното поле со **reflection**
- Најкратко, но зависноста е **скриена** (конструкторот не ја покажува зависноста од надвор)
- Не може `final`
- Тешко тестирање без Spring
- Прифатливо во тест класи (`@SpringBootTest`)

Опционална зависност: @Autowired(required = false)

```java
@Service
public class NotificationService {
    private final NotificationSender sender;
    private AuditLog auditLog;                    // опционално

    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }

    @Autowired(required = false)
    public void setAuditLog(AuditLog auditLog) {
        this.auditLog = auditLog;
    }

    public void notifyUser(String to, String message) {
        sender.send(to, message);
        if (auditLog != null) auditLog.record(to, message);
    }
}
```

- Задолжителното (`sender`) — преку конструктор
- Опционалното (`auditLog`) — преку setter
- Ако нема `AuditLog` бин, setter-от не се повикува и апликацијата **стартува**
- `@Autowired` (default `required = true`) без бин → апликацијата не стартува

Кога @Autowired МОРА да се напише?

Ситуација	Потребно?	Зошто
Еден конструктор (со `final` полиња)	**не**	Spring автоматски го користи
Повеќе конструктори	**да**, на еден	Spring мора да знае кој да го повика
Setter метод	**да**	инаку Spring не го повикува
Поле	**да**	инаку полето останува `null`
`final` поле поставено во конструктор	**не**	вметнувањето е преку конструкторот
Параметар на `@Bean` метод	**не**	Spring автоматски ги вметнува

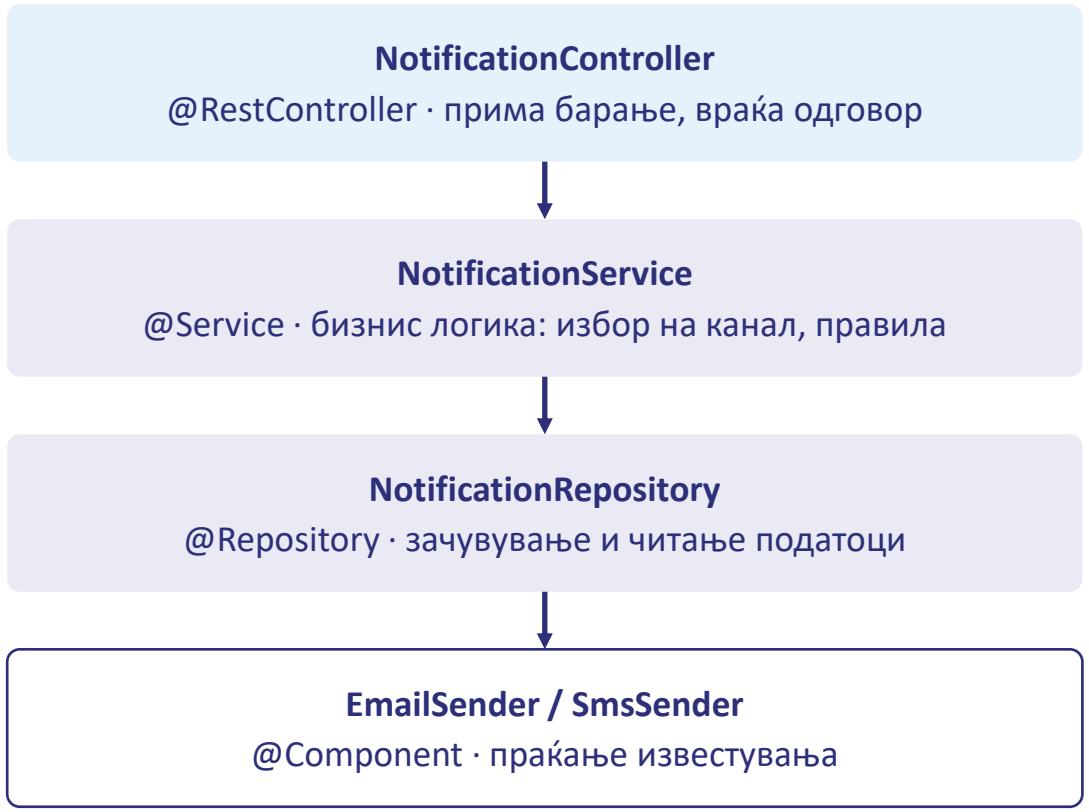
Запомни: `@Autowired` на `final` поле нема смисла — полето се поставува само во конструкторот

Премногу зависности?

```
public NotificationService(EmailSender email,
                           SmsSender sms,
                           PushSender push,
                           NotificationRepository repository,
                           TemplateEngine templates,
                           AuditLog audit,
                           UserDirectory users) {
    ...
}
```

- Конструктор со многу параметри е **знак за проблем**
- Класата веројатно има **премногу одговорности** (S од SOLID)
- Решение: поделете ја на помали класи
- Конструкторот го прави проблемот **видлив** — тоа е добро!

Слоеви на апликацијата



- Секој слој е **бин** со една одговорност
- Зависностите одат **само надолу**
- Секој слој ја добива зависноста преку **конструктор**
- `Notification` (податоци) минува низ сите слоеви, но **не е бин**

Веб слојот (HTTP, URL, JSON) — посебно предавање **Spring MVC**

Одговорности на слоевите

Controller

Го прима барањето
Ги чита влезните податоци
Го повикува сервисот
Враќа одговор
Без бизнис логика

Service

Бизнис правила
Избор на канал (e-mail / SMS)
Проверки и валидации
Не знае за HTTP

Repository

Зачувување и читање
Засега: листа во меморија
Подоцна: база (Spring Data JPA)
Не знае за правилата

Промена во еден слој (пр. база наместо листа) не бара промена во другите

Notification – податоци, не бин

```
public record Notification(
        Long id,
        String to,
        String message,
        String channel) { }

// Користење:
var n = new Notification(1L, "ana@finki.ukim.mk",
                         "Добредојде!", "email");
n.to();         // "ana@finki.ukim.mk"
n.channel();    // "email"
```

- `record` = кратка класа: полиња, конструктор, getters, `equals`, `toString`
- Нема анотација → **не е бин**
- Ги креираме со `new` — сосема во ред!
- Spring не го елиминира `new` — само за компонентите го презема креирањето

NotificationRepository

```
@Repository
public class NotificationRepository {
    private final List<Notification> notifications =
            new CopyOnWriteArrayList<>();
    private final AtomicLong nextId = new AtomicLong(1);

    public Notification save(String to, String msg, String ch) {
        var n = new Notification(nextId.getAndIncrement(),
                                 to, msg, ch);
        notifications.add(n);
        return n;
    }

    public List<Notification> findAll() {
        return List.copyOf(notifications);
    }
}
```

- Одговорност: **пристап до податоци**
- Засега: листа во меморија
- `CopyOnWriteArrayList`, `AtomicLong` — безбедно при паралелни барања (Час 4)
- Нема зависности → Spring прво него го креира

NotificationService

```java
@Service
public class NotificationService {
    private final NotificationSender sender;
    private final NotificationRepository repository;

    public NotificationService(NotificationSender sender,
                               NotificationRepository repository) {
        this.sender = sender;
        this.repository = repository;
    }

    public Notification notifyUser(String to, String message) {
        if (message.isBlank())
            throw new IllegalArgumentException("Празна порака");
        sender.send(to, message);
        return repository.save(to, message, "email");
    }

    public List<Notification> findAll() {
        return repository.findAll();
    }
}
```

- Одговорност: **бизнис логика**
- **Две** зависности преку **еден** конструктор
- Правилото (празна порака) е тука — не во контролерот, не во репозиториумот

NotificationController

```java
@RestController
public class NotificationController {
    private final NotificationService service;

    public NotificationController(NotificationService service) {
        this.service = service;
    }

    @GetMapping("/notify")
    public Notification notifyUser() {
        return service.notifyUser("ana@finki.ukim.mk",
                                  "Добредојде!");
    }

    @GetMapping("/notifications")
    public List<Notification> all() {
        return service.findAll();
    }
}
```

- Одговорност: **комуникација со клиентот**
- Само го повикува сервисот
- Не знае како се праќа ниту каде се чуваат податоците
- Мапирања, параметри, JSON → **Spring MVC**

Редослед на креирање

1. EmailSender
NotificationRepository
(нема зависности)

→

2. NotificationService
(sender, repository)

→

3. NotificationController
(service)

```
// Што Spring прави „зад сценатa“:
var sender = new EmailSender();
var repository = new NotificationRepository();
var service = new NotificationService(sender, repository);
var controller = new NotificationController(service);
```

- Spring го одредува **редоследот**
- Секој бин се креира **еднаш** (Час 4)
- Ние не пишуваме ниту една линија од ова

Два бина од ист тип?

```
@Component
public class EmailSender implements NotificationSender { ... }

@Component
public class SmsSender implements NotificationSender { ... }

@Service
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }
}
```

Прашање: Кој бин ќе го вметне Spring: EmailSender или SmsSender?

- Двата бина **одговараат** на типот `NotificationSender`
- Spring не може сам да одлучи

Грешка: двосмисленост

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Parameter 0 of constructor in com.example.notifications.service.NotificationService
required a single bean, but 2 were found:
    - emailSender: defined in file [.../EmailSender.class]
    - smsSender: defined in file [.../SmsSender.class]

Action:

Consider marking one of the beans as @Primary, updating the consumer to
accept multiple beans, or using @Qualifier to identify the bean that
should be consumed
```

Името на бинот е името на класата со мала прва буква: `EmailSender` → `emailSender`

Решение 1: @Primary

```
@Component
@Primary
public class EmailSender implements NotificationSender {
    ...
}

@Component
public class SmsSender implements NotificationSender {
    ...
}

// NotificationService не се менува:
// се вметнува EmailSender
```

- `@Primary` = „ако има повеќе кандидати, избери го овој“
- Се поставува на **имплементацијата**
- `NotificationService` не се менува
- Добро за **стандардна** имплементација

Решение 2: @Qualifier — на сите три места

Конструктор

```
public NotificationService(
    @Qualifier("smsSender")
    NotificationSender sender) {
  this.sender = sender;
}
```

Setter

```
@Autowired
public void setSender(
    @Qualifier("smsSender")
    NotificationSender s) {
  this.sender = s;
}
```

Поле

```
@Autowired
@Qualifier("smsSender")
private NotificationSender sender;
```

- Кај конструктор: само `@Qualifier` на параметарот (еден конструктор → без `@Autowired`)
- Кај setter и поле: `@Autowired` **и** `@Qualifier`

`@Qualifier("име")` бара бин со точно тоа име · посилно од `@Primary`

Решение 3: име на параметарот / полето

Параметар на конструктор

```
@Service
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(
            NotificationSender smsSender) {  // ← името
        this.sender = smsSender;
    }
}
```

Поле

```
@Service
public class ReminderService {

    @Autowired
    private NotificationSender emailSender;  // ← името
}
```

Ако името на параметарот/полето е исто со името на бинот, Spring го избира — **без анотација**

Како Spring избира бин?



- Кога има повеќе кандидати, Spring гледа по редоследот десно
- Ова е **поедноставено** правило за почеток

1. `@Qualifier` на местото на вметнување
2. `@Primary` на една од имплементациите
3. име на параметарот/полето = име на бинот
4. инаку: грешка (NoUniqueBeanDefinition)

Вметнување на @Bean бинови

```java
@Service
public class NotificationService {
    private final NotificationSender sender;

    // по име на параметарот — без анотација
    public NotificationService(NotificationSender emailSender) {
        this.sender = emailSender;
    }
}

@Service
public class ReminderService {
    @Autowired
    @Qualifier("smsSender")              // експлицитно
    private NotificationSender sender;
}
```

- Бинови од `@Bean` се вметнуваат **исто** како `@Component` бинови
- `emailSender` → име на методот од `SenderConfig`
- `@Qualifier("smsSender")` → истото, експлицитно

Ова е шемата од испитната задача на крајот на презентацијата

@Component или @Bean?

	@Component (и @Service, ...)	**@Bean**
Каде се пишува?	на **класата**	на **метод** во `@Configuration` класа
Кој го креира објектот?	Spring (го повикува конструкторот)	**ние** во методот
Како се наоѓа?	component scan	Spring ги повикува `@Bean` методите
Име на бинот	класа со мала буква: `emailSender`	име на методот
Кога?	**наши** класи	туѓи класи или посебна логика за креирање

Совет: Иста класа **не** ја регистрирај двапати (и `@Component` и `@Bean`)

Резиме на Час 3

- ✓ `@Autowired`: конструктор (опционално), setter и поле (задолжително)
- ✓ Вметнување **преку конструктор** — секогаш; поле — избегнувај
- ✓ **Controller → Service → Repository**: секој слој е бин со една одговорност
- ✓ Повеќе кандидати: `@Qualifier`, `@Primary` или име на параметарот/полето
- ✓ `@Bean` во `@Configuration` — бин со име на методот

Комплетен код · Час 3 (1/5)

model/Notification.java

```
package com.example.notifications.model;

// податоци — НЕ е бин
public record Notification(
        Long id,
        String to,
        String message,
        String channel) {
}
```

repository/NotificationRepository.java

```
package com.example.notifications.repository;

import com.example.notifications.model.Notification;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

@Repository
public class NotificationRepository {
    private final List<Notification> notifications =
            new CopyOnWriteArrayList<>();
    private final AtomicLong nextId = new AtomicLong(1);

    public Notification save(String to, String message, String channel) {
        Notification n = new Notification(
                nextId.getAndIncrement(), to, message, channel);
        notifications.add(n);
        return n;
    }

    public List<Notification> findAll() {
        return List.copyOf(notifications);
    }
}
```

Комплетен код · Час 3 (2/5)

sender/NotificationSender.java · sender/EmailSender.java

```
package com.example.notifications.sender;

public interface NotificationSender {
    void send(String to, String message);
}

// ---------------------------------------------------------
package com.example.notifications.sender;

import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Component;

@Component                          // бин "emailSender"
@Primary                            // стандарден избор
public class EmailSender implements NotificationSender {
    @Override
    public void send(String to, String message) {
        System.out.println("EMAIL до " + to + ": " + message);
    }
}
```

sender/SmsSender.java

```
package com.example.notifications.sender;

import org.springframework.stereotype.Component;

@Component                          // бин "smsSender"
public class SmsSender implements NotificationSender {
    @Override
    public void send(String to, String message) {
        System.out.println("SMS до " + to + ": " + message);
    }
}
```

Комплетен код · Час 3 (3/5)

service/NotificationService.java

```java
package com.example.notifications.service;

import com.example.notifications.model.Notification;
import com.example.notifications.repository.NotificationRepository;
import com.example.notifications.sender.NotificationSender;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class NotificationService {
    private final NotificationSender sender;     // @Primary
    private final NotificationRepository repository;

    public NotificationService(NotificationSender sender,
                               NotificationRepository repository) {
        this.sender = sender;
        this.repository = repository;
    }

    public Notification notifyUser(String to, String message) {
        if (message.isBlank())
            throw new IllegalArgumentException("Празна порака");
        sender.send(to, message);
        return repository.save(to, message, "email");
    }

    public List<Notification> findAll() {
        return repository.findAll();
    }
}
```

service/ReminderService.java

```java
package com.example.notifications.service;

import com.example.notifications.model.Notification;
import com.example.notifications.repository.NotificationRepository;
import com.example.notifications.sender.NotificationSender;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

@Service
public class ReminderService {
    private final NotificationSender sender;     // smsSender
    private final NotificationRepository repository;

    public ReminderService(
            @Qualifier("smsSender") NotificationSender sender,
            NotificationRepository repository) {
        this.sender = sender;
        this.repository = repository;
    }

    public Notification remind(String phone, String message) {
        sender.send(phone, "Потсетник: " + message);
        return repository.save(phone, message, "sms");
    }
}
```

Комплетен код · Час 3 (4/5)

controller/NotificationController.java

```
package com.example.notifications.controller;

import com.example.notifications.model.Notification;
import com.example.notifications.service.NotificationService;
import com.example.notifications.service.ReminderService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController                        // мапирањата детално -> Spring MVC
public class NotificationController {
    private final NotificationService notifications;
    private final ReminderService reminders;

    public NotificationController(NotificationService notifications, ReminderService reminders) {
        this.notifications = notifications;
        this.reminders = reminders;
    }

    @GetMapping("/notify")
    public Notification notifyUser() { return notifications.notifyUser("ana@finki.ukim.mk", "Добредојде!"); }

    @GetMapping("/remind")
    public Notification remind() { return reminders.remind("070123456", "Колоквиум во понеделник"); }

    @GetMapping("/notifications")
    public List<Notification> all() { return notifications.findAll(); }
}
```

Комплетен код · Час 3 (5/5)

Варијанти на вметнување во NotificationService

```
// --- setter injection ---
@Service
public class NotificationService {
    private NotificationSender sender;

    @Autowired
    public void setSender(
            @Qualifier("smsSender") NotificationSender sender) {
        this.sender = sender;
    }
}

// --- field injection ---
@Service
public class NotificationService {
    @Autowired
    @Qualifier("smsSender")
    private NotificationSender sender;
}

// --- по име на полето (без @Qualifier и без @Primary) ---
@Service
public class NotificationService {
    @Autowired
    private NotificationSender smsSender;
}
```

config/SenderConfig.java (наместо @Component на senders)

```
package com.example.notifications.config;

import com.example.notifications.sender.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class SenderConfig {

    @Bean                                   // бин "emailSender"
    public NotificationSender emailSender() {
        return new EmailSender();
    }

    @Bean                                   // бин "smsSender"
    public NotificationSender smsSender() {
        return new SmsSender();
    }
}

// Во овој случај EmailSender и SmsSender
// НЕМААТ @Component (инаку би имало дупли бинови).
// Изборот: по име на параметарот или @Qualifier.
```

Опсег, конфигурација и тестирање

Колку инстанци? Од каде вредностите? Како тестираме?

Цели

Потсетник

- `@Autowired`: конструктор, setter, поле
- Controller → Service → Repository
- `Notification` е податок, не бин
- `@Primary`, `@Qualifier`, име
- `@Bean` во `@Configuration`

Денес ќе научиме

1. Колку инстанци креира Spring: **опсег (scope)**
2. Зошто сервисите не чуваат **состојба**
3. **Животен циклус**: `@PostConstruct`, `@PreDestroy`
4. `application.properties`, `@Value`, **профили**
5. **Тестирање**: зошто DI го олеснува

Прашање: колку инстанци?

```java
@Service
public class NotificationStats {
    private int sent = 0;

    public int increment() {
        return ++sent;
    }

    public int sent() {
        return sent;
    }
}
```

```java
@Service
public class NotificationService {
    private final NotificationSender sender;
    private final NotificationStats stats;

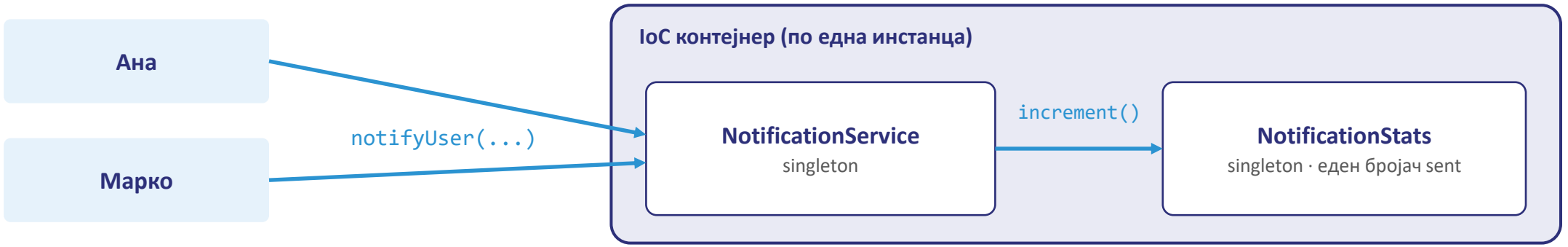
    public NotificationService(NotificationSender sender,
                               NotificationStats stats) {
        this.sender = sender;
        this.stats = stats;
    }

    public void notifyUser(String to, String message) {
        sender.send(to, message);
        stats.increment();          // ← секое праќање се брои
    }
}
```

Ана → `notifyUser("ana@finki.ukim.mk", "Здраво")` → `stats.increment()`
Марко → `notifyUser("marko@finki.ukim.mk", "Здраво")` → `stats.increment()`

Прашање: Ана и Марко праќаат од различни прелистувачи. Дали секој има **свој** бројач `sent` или делат еден?

Одговор: singleton



#	Кој	Што се извршува	sent
1	**Ана**	`notifyUser("ana@finki.ukim.mk", …)` → `stats.increment()`	1
2	**Марко**	`notifyUser("marko@finki.ukim.mk", …)` → `stats.increment()`	2
3	**Ана**	`notifyUser("ana@finki.ukim.mk", …)` → `stats.increment()`	3
4	**Марко**	`GET /stats` → `stats.sent()`	**3** — ги брои и пораките на Ана

Стандардно секој бин е **singleton**: една инстанца за сите корисници и барања → Ана и Марко го делат **истиот** бројач

Состојба во singleton бинови

❌ Опасно

```
@Service
public class NotificationService {

    private String currentRecipient;     // заедничко за СИТЕ!

    public void prepare(String to) {
        currentRecipient = to;
    }

    public void send(String message) {
        sender.send(currentRecipient, message);
    }
}
```

- Марко може да го смени примачот меѓу `prepare()` и `send()` на Ана
- Ана добива туѓа порака — или обратно

Правило: сервисите треба да бидат **без состојба** (stateless)
сè што е потребно доаѓа преку параметри: `notifyUser(to, message)`
трајните податоци — во репозиториум / база

Две барања → ист објект → изгубено зголемување

Ана и Марко праќаат известување **во ист момент** → две нишки истовремено ја извршуваат `stats.increment()` на **истиот** објект

Чекор	Ана: `increment()`	Марко: `increment()`	sent
1	1. чита sent → 5		5
2		1. чита sent → 5	5
3	2–3. пресметува 5 + 1, запишува 6		6
4		2–3. пресметува 5 + 1, запишува 6	**6** (очекувано 7)

```
// return ++sent;  всушност е три чекори:
int tmp = sent;      // 1. чита
tmp = tmp + 1;       // 2. пресметува
sent = tmp;          // 3. запишува
return tmp;
```

```
// Решение: атомично зголемување
private final AtomicInteger sent = new AtomicInteger();

public int increment() {
    return sent.incrementAndGet();  // неделиво
}
```

Марко ја чита старата вредност пред Ана да ја запише новата → едно известување не е избројано. Со `AtomicInteger`: Ана 6, Марко 7.

Опсези (scopes)

Опсег	Колку инстанци?	Пример	Кај сервлетите
singleton (стандарден)	една за целиот контејнер	`NotificationService`	≈ application атрибут
prototype	нова секогаш кога се бара бинот	`NotificationDraft`	—
request	една по HTTP барање	id на барањето за логови	request атрибут
session	една по HTTP сесија	преференци на корисникот	session атрибут
application	една по ServletContext	заеднички податоци	ServletContext атрибут

`@Scope("prototype")` · `@RequestScope` · `@SessionScope` — на класата, заедно со `@Component`

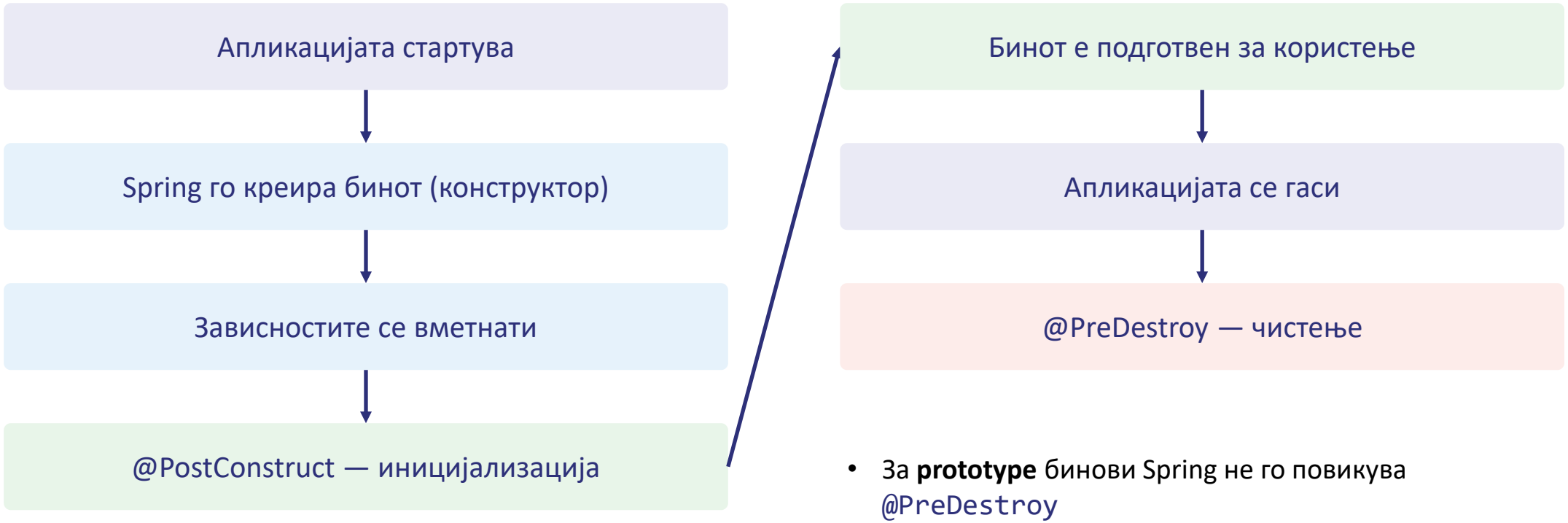
Prototype ≠ request

```
@Component
@Scope("prototype")
public class NotificationDraft {
    private final List<String> lines = new ArrayList<>();
    public void add(String line) { lines.add(line); }
}

NotificationDraft a = context.getBean(NotificationDraft.class);
NotificationDraft b = context.getBean(NotificationDraft.class);
System.out.println(a == b);   // false
```

- Нова инстанца секогаш кога **се бара** бинот од контејнерот
- Вметнат во singleton → се креира **само еднаш**
- **request** е поврзан со HTTP барање, **prototype** — не

Животен циклус на бин



- За **prototype** бинови Spring не го повикува `@PreDestroy`

@PostConstruct и @PreDestroy

```
@Component
public class EmailSender implements NotificationSender {

    @PostConstruct
    public void connect() {
        System.out.println("Поврзување со SMTP сервер");
    }

    @PreDestroy
    public void disconnect() {
        System.out.println("Затворање на врската");
    }

    @Override
    public void send(String to, String message) { ... }
}
```

- `@PostConstruct`: **по** вметнувањето — поврзување, почетни податоци, проверки
- `@PreDestroy`: пред гаснење — затворање ресурси
- Пакет: `jakarta.annotation`
- Повеќето сервиси не им се потребни овие методи

Не хардкодирај вредности

Не: вредностите во кодот

```
@Service
public class NotificationService {

    public Notification notifyUser(String to, String msg) {
        if (msg.length() > 160) { ... }    // magic number
        sender.send(to, msg + "\n– Тимот на ФИНКИ");
        ...
    }
}
```

Да: вредностите надвор од кодот

```
# application.properties
notification.max-length=160
notification.signature=Тимот на ФИНКИ

// Java
@Value("${notification.max-length}")
private int maxLength;
```

Промена на ограничување или потпис не бара промена и повторно компајлирање на кодот

application.properties: каде е и како изгледа

```
# src/main/resources/application.properties

# --- вградени Spring Boot својства ---
spring.application.name=notifications
server.port=8080
spring.profiles.active=dev

# --- наши својства ---
app.name=FINKI Notifications
notification.max-length=160
notification.default-channel=email
notification.signature=Тимот на ФИНКИ
notification.channels=email,sms
notification.enabled=true
```

- Локација: `src/main/resources/`
- Spring Boot ја чита **автоматски** при стартување
- Формат: `клуч=вредност`; `#` = коментар
- Точки за групирање: `notification.max-length`
- Препорачано: мали букви и `-` (kebab-case)

Името е важно: `application.properties`. Друго име (`app.properties`) не се чита автоматски.

.properties или .yml — иста конфигурација

application.properties

```
server.port=8080
notification.max-length=160
notification.default-channel=email
notification.channels=email,sms
```

application.yml

```
server:
  port: 8080
notification:
  max-length: 160
  default-channel: email
  channels:
    - email
    - sms
```

Користи **еден** формат во проект. Кај YAML вовлекувањето (празни места) е важно.

Својства што Spring Boot ги разбира сам

Својство	Пример	Ефект
`server.port`	`8081`	порта на вградениот Tomcat
`server.servlet.context-path`	`/api`	сите URL добиваат префикс `/api`
`spring.application.name`	`notifications`	име на апликацијата (логови, мониторинг)
`spring.profiles.active`	`dev`	кој профил е активен
`logging.level.<пакет>`	`DEBUG`	ниво на логирање за пакет
`spring.datasource.url`	`jdbc:...`	база на податоци (предавање за Spring Data)

Наши својства за известувањата

```
app.name=FINKI Notifications
notification.max-length=160
notification.default-channel=email
notification.signature=Тимот на ФИНКИ
notification.channels=email,sms
notification.enabled=true
```

Клуч	Тип во Java
notification.max-length	int
notification.default-channel	String
notification.signature	String
notification.channels	String[] / List
notification.enabled	boolean

Конверзија: Во датотеката сè е текст; Spring ја претвора вредноста во типот на полето/параметарот

Читање: @Value (поле, конструктор, setter) или групно со @ConfigurationProperties

@Value на поле

```
@Service
public class NotificationService {

    @Value("${notification.signature}")
    private String signature;

    @Value("${notification.enabled}")
    private boolean enabled;

    public Notification notifyUser(String to, String msg) {
        if (!enabled) throw new IllegalStateException("Исклучено");
        sender.send(to, msg + "\n– " + signature);
        ...
    }
}
```

- Најкраток начин
- Spring го поставува полето **по** конструкторот
- Полето не може да биде `final`
- Во конструкторот вредноста е уште `null`!

Исти предности и недостатоци како `@Autowired` на поле

@Value во конструктор

```
@Service
public class NotificationService {
    private final NotificationSender sender;
    private final NotificationRepository repository;
    private final int maxLength;

    public NotificationService(
            NotificationSender sender,
            NotificationRepository repository,
            @Value("${notification.max-length:160}") int maxLength) {
        this.sender = sender;
        this.repository = repository;
        this.maxLength = maxLength;
    }
}
```

- `@Value` на **параметарот** на конструкторот
- Полето е `final` и достапно веднаш
- Се меша со обични зависности (бинови)
- Вредноста што следува по двете точки (:) е предефинирана и се користи ако не е поставена вредност за својството
- Тест: `new NotificationService(fake, repo, 10)`

Препорачан начин — исто како вметнување преку конструктор за бинови

@Value на setter

```
@Service
public class NotificationService {
    private String defaultChannel;

    @Value("${notification.default-channel:email}")
    public void setDefaultChannel(String defaultChannel) {
        this.defaultChannel = defaultChannel;
    }

    public Notification notifyUser(String to, String msg,
                                   String channel) {
        String ch = (channel == null) ? defaultChannel : channel;
        ...
    }
}
```

- `@Value` на setter — Spring го повикува при креирање
- Не е потребен `@Autowired`
- Корисно ако сакаме логика/валидација во setter-от

Три места за `@Value` — исти како трите места за `@Autowired`

@Value: што точно се вметнува?

Израз	Тип	Резултат
`@Value("${app.name}")`	`String`	`FINKI Notifications` — вредност од датотеката
`@Value("app.name")`	`String`	`app.name` — **буквален текст**, без `${}`!
`@Value("${app.title:Notifications}")`	`String`	`Notifications` — предефинирна вредност (ако не е поставено app.title
`@Value("${notification.max-length}")`	`int`	`160` — како број (автоматска конверзија)
`@Value("${notification.enabled}")`	`boolean`	`True`
`@Value("${notification.channels}")`	`String[]`	`{"email", "sms"}` — поделено со запирка при дефинирањето на својството
`@Value("#{2 * 80}")`	`int`	`160` — SpEL израз

Запомни: `${...}` = својство од конфигурацијата · `#{...}` = SpEL израз · без нив = буквален текст

SpEL: изрази во @Value

```java
@Service
public class NotificationService {

    // 1. аритметика: 160 - 20 знаци за потпис
    @Value("#{${notification.max-length} - 20}")
    private int bodyLength;                    // → 140

    // 2. метод на String (текстот оди во '...')
    @Value("#{'${notification.default-channel}'.toUpperCase()}")
    private String channel;                    // → "EMAIL"

    // 3. услов: дали SMS е меѓу каналите?
    @Value("#{'${notification.channels}'.contains('sms')}")
    private boolean smsEnabled;                // → true

    // 4. тернарен оператор
    @Value("#{${notification.enabled} ? 'вклучено' : 'исклучено'}")
    private String status;                     // → "вклучено"

    // 5. статички метод T(...) + текст + својство
    @Value("© #{T(java.time.Year).now().value} ${app.name}")
    private String footer;                     // → "© 2026 FINKI Notifications"
}
```

- `Spring Expression Language`
- `#{...}` → Spring го пресметува изразот при креирање на бинот
- `${...}` внатре во `#{...}` се заменува прво
- Текст во изразот → во `'...'`
- `T(Класа)` → статички методи од Java
- **Совет:** само кратки изрази; логиката оди во Java код

Запомни: `${...}` = прочитај својство · `#{...}` = пресметај израз

Чести грешки со @Value

Конзола: клучот не постои

```
APPLICATION FAILED TO START

Could not resolve placeholder
'notification.max-lenght' in value
"${notification.max-lenght}"
             ^ печатна грешка
```

Други замки

```
// 1. буквален текст наместо вредност
@Value("notification.signature") String s;

// 2. објект создаден со new → нема вметнување
var s = new NotificationService(...);
// полињата со @Value остануваат null / 0

// 3. static поле → Spring го игнорира
@Value("${app.name}") static String NAME;  // null

// 4. променета датотека додека работи
//    → нема ефект без рестарт
```

Клуч без стандардна вредност што го нема → апликацијата **не стартува** (грешката се гледа веднаш)

@ConfigurationProperties: групирани поставки

```
@ConfigurationProperties(prefix = "notification")
public record NotificationProperties(
        int maxLength,           // notification.max-length
        String defaultChannel,   // notification.default-channel
        String signature,
        List<String> channels,
        boolean enabled) { }

@SpringBootApplication
@ConfigurationPropertiesScan      // ги наоѓа и регистрира
public class NotificationsApplication { ... }

// користење – како секој друг бин:
public NotificationService(NotificationProperties props, ...)
```

- Сите `notification.*` клучеви → **еден** објект
- `max-length` → `maxLength` (relaxed binding)
- Типизирано, полесно за одржување од многу `@Value`
- Се вметнува преку конструктор

@Value или @ConfigurationProperties?

	@Value	**@ConfigurationProperties**
Колку вредности	една по една	цела група со префикс
Каде	поле, конструктор, setter	посебна класа / record
SpEL (`#{...}`)	да	не
`max-length` ↔ `maxLength`	делумно	целосно
Тестирање	секоја вредност посебно	еден објект

Правило: Почни со `@Value`; кога клучевите со ист префикс ќе станат многу — `@ConfigurationProperties`

Од каде доаѓа вредноста и кој победува?

Приоритет	Извор	Пример за notification.max-length
1 (највисок)	аргумент од командна линија	`--notification.max-length=100`
2	променлива на околината	`export NOTIFICATION_MAXLENGTH=120`
3	`application-{profile}.properties`	`application-prod.properties: 140`
4	`application.properties`	`160`
5 (најнизок)	предефинирана вредност во `@Value`	`${notification.max-length:200}`

Пример: Активен `prod`, без аргументи и променливи → `140`

Профили: зошто?

dev

Локален развој
Известувања само во конзола
Порта 8081

test

Автоматски тестови
Лажни senders
Контролирана конфигурација

prod

Вистински SMTP сервер
Порта 80
Лозинки од околината

Истиот код — различна **конфигурација** и различни **бинови**

Конфигурација по профил

application.properties

```
spring.profiles.active=dev
server.port=8080
notification.max-length=160
notification.signature=Тимот на ФИНКИ
```

application-dev.properties

```
server.port=8081
notification.signature=[DEV] Тимот на ФИНКИ
```

application-prod.properties

```
server.port=80
notification.max-length=140
```

Активирање на профил

1. `spring.profiles.active=dev` во `application.properties`
2. при стартување: `--spring.profiles.active=prod`
3. IntelliJ: Run Configuration → Active profiles

Профилната вредност го препокрива истиот клуч од `application.properties`

Предвиди: кои вредности важат кога е активен dev?

A `8080` и „Тимот на ФИНКИ“

B `8081` и „[DEV] Тимот на ФИНКИ“

C `80` и „Тимот на ФИНКИ“

D Апликацијата не стартува — клучот е дефиниран двапати

Прашање: Со датотеките од претходниот слајд: колку е `server.port` и кој потпис се праќа? А колку е `notification.max-length`?

@Profile – различни бинови

dev: само печати

```
@Component("emailSender")
@Profile("dev")
public class ConsoleEmailSender
        implements NotificationSender {

    public void send(String to, String msg) {
        System.out.println(
            "[DEV] EMAIL до " + to + ": " + msg);
    }
}
```

prod: вистински SMTP

```
@Component("emailSender")
@Profile("prod")
public class EmailSender
        implements NotificationSender {

    public void send(String to, String msg) {
        // поврзување со SMTP сервер ...
    }
}
```

Двата се со име `emailSender`, но во секој профил постои **само еден** → сервисот не се менува

Зошто DI го олеснува тестирањето?

❌ **Без DI:** класата сама си ја креира зависноста

```
@Service
public class NotificationService {
    private final NotificationSender sender =
            new EmailSender();     // секогаш вистински

    public void notifyUser(String to, String msg) {
        sender.send(to, msg);
    }
}
// тест → праќа вистински e-mail!
```

✓ **Со DI:** зависноста доаѓа однадвор

```
@Service
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }
    ...
}
// Spring → EmailSender  ·  тест → FakeSender
```

Unit тест

Тестира една класа — без Spring, без мрежа, без база. Трае милисекунди и дава ист резултат секој пат.

Лажен (fake) објект

Наша едноставна имплементација на интерфејсот: не праќа ништо, само памети што добила.

Ние сме „контејнерот“

Тестот со new го креира сервисот и му ги предава зависностите — тоа што инаку го прави Spring.

Тука: само врската DI ↔ тестирање. JUnit, Mockito, @SpringBootTest — детално на посебното предавање за тестирање.

Unit тест без Spring: чекор по чекор

```java
class NotificationServiceTest {

    // лажен sender: не праќа, само памети
    static class FakeSender implements NotificationSender {
        String lastTo, lastMessage;
        public void send(String to, String message) {
            lastTo = to;
            lastMessage = message;
        }
    }

    @Test
    void notifyUserSendsAndStores() {
        // 1. Подготви
        var fake = new FakeSender();
        var service = new NotificationService(
                fake, new NotificationRepository(), 160);
        service.setDefaultChannel("email");

        // 2. Изврши
        Notification n = service.notifyUser("ana@finki.ukim.mk", "Здраво");

        // 3. Провери
        assertEquals("ana@finki.ukim.mk", fake.lastTo);
        assertTrue(fake.lastMessage.startsWith("Здраво"));
        assertEquals(1L, n.id());
    }
}
```

@Test

JUnit го извршува методот како тест (JUnit е во spring-boot-starter-test).

FakeSender

Наместо EmailSender. Ја памети пораката за да можеме да провериме што е пратено.

1. Подготви

Ние ја играме улогата на Spring: new + зависностите преку конструкторот; setter-от го повикуваме сами.

2. Изврши

Повикуваме еден метод — тоа што го тестираме.

3. Провери

assertEquals(очекувано, добиено), assertTrue(услов). Ако не важи → тестот паѓа (црвено).

Spring не се стартува → полето со @Value (signature) останува null. Само конструкторот гарантира дека зависностите се поставени.

Што е најважно во пракса?

Поим	Важност	Каде ќе го користиме
Бин, IoC контејнер, вметнување преку конструктор	висока	секоја класа во апликацијата
`@Service`, `@Repository`, `@RestController`	висока	слоеви на апликацијата
Singleton и сервиси без состојба	висока	избегнување грешки со корисници
`@Autowired`, `@Qualifier`, `@Primary`	средна	повеќе имплементации, туѓ код
`application.properties`, `@Value`, `@Profile`	средна	база, лозинки, околини
Тестирање со DI	средна	unit тестови на сервисите
Детален животен циклус, XML, BeanFactory	ниска	додатен материјал

Резиме на темата

Што е бин?
Објект што го креира и со кој управува Spring

Што е IoC?
Spring ја презема контролата за креирање и поврзување

Што е DI?
Предавање на зависностите, најчесто преку конструктор

Кога мора @Autowired?
Повеќе конструктори, setter, поле

Како Spring знае што да креира?
Component scan (`@Component`, ...) и `@Bean` методи

Нов сервис за секое барање?
Не — стандардно е singleton

@Value("app.name")?
Буквален текст — потребно е `${app.name}`

Два бина од ист тип?
`@Qualifier`, `@Primary` или име

Што следува?

Денес
бинови и DI

→

Spring MVC
контролери, барања, одговори

→

Spring Data JPA
NotificationRepository од база

→

Spring Security
кој смее да праќа

Структурата **Controller → Service → Repository** и DI остануваат во сите следни теми

Комплетен код · (1/6)

application.properties

```
# src/main/resources/application.properties
spring.application.name=notifications
server.port=8080
spring.profiles.active=dev

app.name=FINKI Notifications
notification.max-length=160
notification.default-channel=email
notification.signature=Тимот на ФИНКИ
notification.channels=email,sms
notification.enabled=true
```

application-dev.properties · application-prod.properties

```
# src/main/resources/application-dev.properties
server.port=8081
notification.signature=[DEV] Тимот на ФИНКИ


# src/main/resources/application-prod.properties
server.port=80
notification.max-length=140
notification.smtp-host=smtp.finki.ukim.mk


# стартување со prod профил:
# java -jar notifications.jar --spring.profiles.active=prod
```

Комплетен код · (2/6)

service/NotificationService.java

```
package com.example.notifications.service;

import com.example.notifications.model.Notification;
import com.example.notifications.repository.NotificationRepository;
import com.example.notifications.sender.NotificationSender;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class NotificationService {
    private final NotificationSender sender;
    private final NotificationRepository repository;
    private final int maxLength;

    @Value("${notification.signature}")                          // @Value на поле
    private String signature;

    private String defaultChannel;
    public NotificationService(NotificationSender emailSender,       // по име: бином "emailSender"
                               NotificationRepository repository,    // @Value во конструктор
                               @Value("${notification.max-length:160}") int maxLength) {
        this.sender = emailSender;
        this.repository = repository;
        this.maxLength = maxLength;
    }
    // setter и методите -> следниот слајд
```

Комплетен код · Час 4 (3/6)

service/NotificationService.java — setter и методи

```
    @Value("${notification.default-channel:email}")                    // @Value на setter
    public void setDefaultChannel(String defaultChannel) {
        this.defaultChannel = defaultChannel;
    }

    public Notification notifyUser(String to, String message) {
        if (message.isBlank()) {
            throw new IllegalArgumentException("Празна порака");
        }
        if (message.length() > maxLength) {
            throw new IllegalArgumentException("Пораката е подолга од " + maxLength + " знаци");
        }
        sender.send(to, message + "\n— " + signature);
        return repository.save(to, message, defaultChannel);
    }

    public List<Notification> findAll() {
        return repository.findAll();
    }
}

// NotificationRepository, Notification, ReminderService и NotificationController
// се исти како во Час 3.
```

Комплетен код · Час 4 (4/6)

sender/ConsoleEmailSender.java · sender/SmsSender.java

```java
package com.example.notifications.sender;

import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Component;

@Component("emailSender")
@Profile("dev")
public class ConsoleEmailSender implements NotificationSender {
    @Override
    public void send(String to, String msg) {
        System.out.println("[DEV] EMAIL до " + to + ": " + msg);
    }
}

// ------------------------------------------------------------
package com.example.notifications.sender;
import org.springframework.stereotype.Component;

@Component                                  // во сите профили
public class SmsSender implements NotificationSender {
    @Override
    public void send(String to, String message) {
        System.out.println("SMS до " + to + ": " + message);
    }
}
```

sender/EmailSender.java (prod, со lifecycle)

```java
package com.example.notifications.sender;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Component;

@Component("emailSender")
@Profile("prod")
public class EmailSender implements NotificationSender {
    private final String smtpHost;

    public EmailSender(
            @Value("${notification.smtp-host}") String host) {
        this.smtpHost = host;
    }

    @PostConstruct
    void connect() { System.out.println("Поврзување со " + smtpHost); }
    @PreDestroy
    void disconnect() { System.out.println("Затворање: " + smtpHost); }

    @Override
    public void send(String to, String message) {
        System.out.println("EMAIL (" + smtpHost + ") до "
                + to + ": " + message);
    }
}
```

Комплетен код · (5/6)

config/NotificationProperties.java · NotificationsApplication.java

```java
package com.example.notifications.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import java.util.List;

@ConfigurationProperties(prefix = "notification")
public record NotificationProperties(int maxLength,
                                     String defaultChannel,
                                     String signature,
                                     List<String> channels,
                                     boolean enabled) { }

// ------------------------------------------------------------
package com.example.notifications;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

@SpringBootApplication
@ConfigurationPropertiesScan
public class NotificationsApplication {
    public static void main(String[] args) {
        SpringApplication.run(NotificationsApplication.class, args);
    }
}
```

controller/InfoController.java

```java
package com.example.notifications.controller;

import com.example.notifications.config.NotificationProperties;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class InfoController {

    @Value("${app.name}")
    private String appName;

    private final NotificationProperties props;

    public InfoController(NotificationProperties props) {
        this.props = props;
    }

    @GetMapping("/info")
    public String info() {
        return appName
                + " | max " + props.maxLength()
                + " | канали " + props.channels()
                + " | потпис " + props.signature();
    }
}
```

Комплетен код · (6/6)

service/NotificationStats.java

```java
package com.example.notifications.service;

import org.springframework.stereotype.Service;
import java.util.concurrent.atomic.AtomicInteger;

@Service                                    // singleton
public class NotificationStats {
    private final AtomicInteger sent = new AtomicInteger();

    public int increment() {
        return sent.incrementAndGet();
    }

    public int sent() {
        return sent.get();
    }
}

// во NotificationService.notifyUser(...):
//     stats.increment();
// (NotificationStats се додава како уште една
//  зависност во конструктором)
```

controller/StatsController.java

```java
package com.example.notifications.controller;

import com.example.notifications.service.NotificationStats;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class StatsController {
    private final NotificationStats stats;

    public StatsController(NotificationStats stats) {
        this.stats = stats;
    }

    @GetMapping("/stats")
    public int stats() {
        return stats.sent();
    }
}
```

Пример од испит

Со **минимален број анотации** пополнете ги празните полиња за Spring контејнерот успешно да се иницијализира. Користете ја семантички најсоодветната анотација од листата. Каде не треба — „ниту едно“; кај две полиња — по име растечки (без @), „ниту едно“ прво.

```
 1
class StudentRepositoryConfig {
     2
    public StudentRepository fcseStudents() {
        // return instance for FCSE
    }
     3
    public StudentRepository tmfStudents() {
        // return instance for TMF
    }
}

 4
class UserRepository { /* ... */ }

 14
class TMFStudentsService {
     15
    private UserRepository repository;
     16          17
    private StudentRepository tmfStudents;
}
```

```
 5
class FCSEStudentService {
     6
    private final UserRepository repository;
     7           8
    private final StudentRepository x;
     9
    private String appName;

    public FCSEStudentService(
         10          11
        UserRepository repository,
         12          13
        StudentRepository fcseStudents) {
        // ...
    }
}
```

[18] application.properties: app.name=FCSE; application-tmf.properties: app.name=TMF. Која вредност добива appName?

FCSE · TMF · ниту едно од наведените

Понудени анотации

важат за сите полиња

ниту едно

```
@Autowired
@Bean
@Component
@ComponentScan
@Configuration
@Controller
@Inject
@Primary
@Profile("tmf")
@Qualifier("app.name")
@Qualifier("fcseStudents")
@Qualifier("tmfStudents")
@Repository
@Resource
@Service
@Value
@Value("${app.name}")
@Value("app.name")
```

Пример од испит: решение

```
@Configuration   // [1]
class StudentRepositoryConfig {
    @Bean   // [2]
    public StudentRepository fcseStudents() { ... }

    @Bean   // [3]
    public StudentRepository tmfStudents() { ... }
}

@Repository   // [4]
class UserRepository { ... }

@Service   // [14]
class TMFStudentsService {
    @Autowired   // [15]
    private UserRepository repository;

    @Autowired   // [16] ниту едно, [17] @Autowired
    private StudentRepository tmfStudents;       // по име на полето
}
```

```
@Service   // [5]
class FCSEStudentService {

    private final UserRepository repository;   // [6] ниту едно

    private final StudentRepository x;   // [7][8] ниту едно

    @Value("${app.name}")   // [9]
    private String appName;

    public FCSEStudentService(
        UserRepository repository,   // [10][11] ниту едно
        StudentRepository fcseStudents   // [12][13] ниту едно
    ) { ... }   // по име на параметаром
}

// [18] appName = "FCSE"
//      профилот tmf не е активен
```

Додатен материјал

За љубопитните — не е дел од основното предавање

XML конфигурација (историски)

```xml
<beans xmlns="http://www.springframework.org/schema/beans">

    <bean id="emailSender"
          class="com.example.notifications.EmailSender"/>

    <bean id="notificationService"
          class="com.example.notifications.NotificationService">
        <constructor-arg ref="emailSender"/>
    </bean>
</beans>
```

- Пред анотациите сè се конфигурираше во XML
- Контејнер: `ClassPathXmlApplicationContext`
- Ќе го сретнете во стари проекти
- Денес: анотации + Java конфигурација

BeanFactory и ApplicationContext

	BeanFactory	**ApplicationContext**
Што е?	основен интерфејс на контејнерот	го проширува BeanFactory
Креирање singleton бинови	кога ќе се побараат	при стартување
Дополнително	—	настани, интернационализација, интеграција со AOP
Употреба	ретко, директно	**секогаш** во апликации

Имплементации: `AnnotationConfigApplicationContext`, `ClassPathXmlApplicationContext`, `AnnotationConfigServletWebServerApplicationContext`

SOLID: L и I

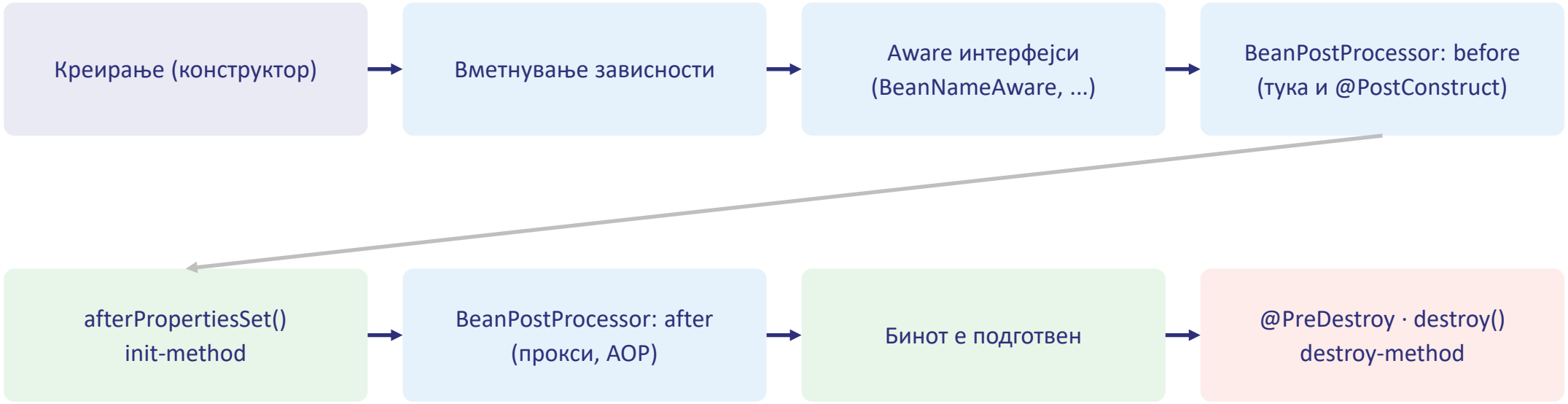
```
// Погрешно: секој sender мора да праќа и прилог
interface NotificationSender {
    void send(String to, String message);
    void sendWithAttachment(String to, byte[] file);  // SMS не може!
}

// Правилно: поделени интерфејси
interface NotificationSender {
    void send(String to, String message);
}
interface AttachmentSender {
    void sendWithAttachment(String to, byte[] file);
}

class EmailSender implements NotificationSender, AttachmentSender { ... }
class SmsSender   implements NotificationSender { ... }
```

- **L – Liskov Substitution:** секоја имплементација може да ја замени друга без изненадувања (пр. без да фрла исклучок)
- **I – Interface Segregation:** помали интерфејси — класата имплементира само тоа што ѝ треба

Детален животен циклус



За почеток доволно е: **креирање → вметнување → @PostConstruct → користење → @PreDestroy**

Ресурси

- **Spring Framework – IoC контејнер:** docs.spring.io/spring-framework/reference/core/beans.html
- **Dependency Injection / @Autowired:** docs.spring.io/spring-framework/reference/core/beans/annotation-config/autowired.html
- **Spring Boot – референца:** docs.spring.io/spring-boot/reference/
- **Надворешна конфигурација:** docs.spring.io/spring-boot/reference/features/external-config.html
- **Профили:** docs.spring.io/spring-boot/reference/features/profiles.html
- **Генератор на проекти:** start.spring.io